Software Test Plan for Traffic Management System

Project: Traffic Management System (Simulation)
Version: 1.0
Authors: Aarya and Vainik
Date: 11-10-2026
Status: Draft for review; tests not executed

1. Introduction

Purpose: This Software Test Plan defines the scope, approach, resources, and acceptance evidence for testing the Traffic Management System (TMS), a C++17, time-stepped traffic simulation. It gives the team a common plan for checking that the implementation satisfies the approved SRS.

Scope: Testing covers scenario configuration, vehicle generation and movement, queues, road networks, signal control, emergency priority, routing, congestion, incidents, metrics, reporting, reproducibility, and the specified quality attributes. The system is a local simulation and does not control physical traffic infrastructure.

References: Traffic Management System SRS, Version 1.0, dated 09-10-2026; Software Engineering Mini-Project Deliverables Part-1; supplied Software Test Plan template.

Definitions: TMS (Traffic Management System); SRS (Software Requirements Specification); STP (Software Test Plan); FR/NFR (Functional/Non-functional Requirement); RTM (Requirements Traceability Matrix); CSV (Comma-Separated Values); simulation tick (one discrete update step); RAII (Resource Acquisition Is Initialization).

2. Test Items

- Scenario configuration and loading, including roads, intersections, capacities, duration, and vehicle-generation settings.
- Road-network representation, routing, lane queues, vehicle generation, identifiers, and movement.
- Signal phases, timers, adaptive timing, and emergency-priority handling.
- Congestion detection, incident injection, rerouting, and traffic-management responses.
- Simulation metrics, text/CSV report generation, deterministic seeds, and build/test interfaces.
- Console menu and command behavior, input validation, and error handling across the above modules.

3. Features to be Tested

The planned coverage includes every requirement in the SRS. Detailed procedures and the requirement-to-test mapping appear in Section 13.

- FR-01 through FR-04: scenario loading, vehicle generation and attributes, and entry-queue capacity.
- FR-05 through FR-08: valid signal phases, safe transitions, adaptive green timing, and emergency priority.
- FR-09 through FR-12: road-network data, vehicle movement, downstream capacity, and route selection.
- FR-13 through FR-18: congestion, incidents, response/rerouting, metrics, reports, and deterministic simulation.
- NFR-01 through NFR-02: module interfaces, controlled shared state, and independent deterministic unit testing.
- NFR-03 through NFR-06: benchmark performance, safe rejection of invalid configuration, Linux/C++17/CMake portability, and console/report usability.
- NFR-07 through NFR-10: input/security robustness, source/test traceability, code quality, and seed recording/reproducibility.

4. Features Not to be Tested

- Control of real traffic lights, vehicles, roads, or other physical traffic infrastructure.
- Live GPS, CCTV, sensor, or external traffic-data integration.
- Machine-learning-based traffic prediction or optimization.
- Detailed vehicle physics beyond the discrete movement rules in the SRS.
- External banking, payment, cloud, or network services; they are outside this standalone in-process simulation.

5. Test Approach / Strategy

Test Levels and Types

Levels: Unit tests exercise deterministic module methods independently; integration tests check defined interactions among scenario, network, vehicle, signal, incident, metrics, and reporting modules; system tests run complete scenarios through the console interface; acceptance tests use the SRS scenarios and quality attributes with the project evaluator or team reviewer.

Types: Functional, boundary/input-validation, safety, regression, performance, portability/build, usability, reproducibility, and security/robustness testing. Review-based checks are used where an SRS attribute concerns code structure or documentation rather than runtime output.

Test design: Use valid, boundary, and invalid configurations; small deterministic scenarios for expected outcomes; and the SRS benchmark scenario for scale. Seeded input is used wherever repeatability is required. Each test records its SRS requirement ID, input, expected result, actual result, status, and defect reference when applicable.

Entry criteria: A build that configures and compiles with C++17 and CMake on Linux is available; interfaces needed by the planned tests are implemented; a stable test scenario and data are available; and the test runner can execute the relevant checks.

Exit criteria: All planned tests have an execution status and evidence; all high-priority FRs and critical safety/capacity checks pass; no unresolved critical defects remain; applicable NFR tests are reviewed; the benchmark scenario completes and its elapsed time is recorded; and any remaining failures or limitations are documented for acceptance review.

Defect handling: Record each failure with test ID, build/configuration, seed where relevant, reproduction steps, observed and expected results, severity, and evidence. Re-run the failed case and related regression cases after a fix.

5.1 Security Validation

Security validation is scoped to the local console, scenario/configuration inputs, in-process modules, and local report output described by the SRS. The SRS does not define user authentication, network transport, encryption, or a remote service, so banking and TLS checks from the sample template do not apply.

- Submit malformed, missing, out-of-range, negative, and inconsistent file/menu values, including impossible capacities/timings and unknown network references. The program must reject them with clear errors and without a crash or undefined behavior (NFR-04, NFR-07).
- Exercise boundary and malformed input strings through supported file/menu fields; verify validation occurs before values are used and that unsafe C-style string handling is absent in the implementation (NFR-07).
- Exercise file-open/read/write failure paths that the interface supports; verify resources close on success and failure and are managed with RAII (NFR-07).
- Review error output and logs for accidental disclosure of raw internal state or unusable diagnostics; messages should be readable and should not expose sensitive data. No confidential data is specified for this simulation (NFR-06, NFR-07).
- Review critical code paths with compiler warnings and the project-selected static-analysis checks; resolve critical findings before acceptance (NFR-09).

6. Test Environment

Hardware: A standard student laptop. Record CPU, memory, and operating-system version when running the NFR-03 performance benchmark so the elapsed time can be interpreted.

Software: Linux; C++17-compatible compiler; CMake; project executable; local scenario/configuration files; local filesystem for text/CSV output.

Tools: CMake and compiler/build output; the project's unit-test runner; version control for build/source revision; a text editor or CSV reader for output inspection. The project team will identify the test runner and any static-analysis tool used before execution.

Test data: Small deterministic networks and schedules for controlled cases; invalid configuration fixtures for validation; an alternate-route/no-route network for incident cases; and the SRS benchmark of at least four intersections and 500 generated vehicles. Store the seed and configuration with each reproducibility run.

7. Test Schedule

The order below is a planning sequence. Calendar dates for implementation and execution depend on the project coding and integration schedule and are to be confirmed by the team.

- Test case review and scenario preparation: before corresponding module testing.
- Unit and module testing: as deterministic module interfaces become available.
- Integration and regression testing: after connected modules are integrated.
- System, security/robustness, and usability testing: on a stable integrated build.
- Performance and acceptance testing: after the SRS benchmark scenario and reporting path are available.
- Test summary and approval: after execution results and unresolved defects are reviewed.

8. Test Deliverables

- Software Test Plan with the Requirements Traceability Matrix and detailed test cases.
- Deterministic scenario/configuration fixtures and recorded seed values.
- Test execution records and build/performance observations.
- Defect reports with reproduction information and evidence.
- Final test summary covering coverage, results, defects, and acceptance status.

9. Roles and Responsibilities

Role	Name	Responsibility
Test Plan and case owners	Aarya and Vainik	Maintain this plan and RTM; review, execute, and record the planned test cases when the implementation is available.
Module owners	As identified in the SRS	Provide module interfaces, testable builds, fixes, and source-file mappings for implemented requirements.
Project reviewer / evaluator	To be assigned by the team	Review acceptance evidence, open issues, and readiness for sign-off.

10. Risks and Mitigation

Risk	Mitigation
Module interfaces or stable build arrive late.	Prepare small deterministic fixtures and run module tests as each interface becomes available; record blocked cases.
Different modules use inconsistent data or timing assumptions.	Agree data structures, tick semantics, capacity units, and error behavior during integration; add boundary integration cases.

Risk	Mitigation
Performance varies across student machines.	Record machine/build details and elapsed time; use the same benchmark configuration and seed for comparisons.
No test runner or static-analysis command is selected yet.	Confirm the project tools before coding is complete and record exact commands in the execution log.

11. Assumptions & Dependencies

- The system remains a standalone C++17 simulation built with CMake on Linux.
- The console commands, configuration format, and module interfaces will be documented by the implementation team.
- Deterministic scenarios and random seeds can be supplied and the seed used can be printed or stored.
- The test runner and static-analysis tool, if used, will be selected by the project team before execution.
- Source-file names are not specified in the SRS; they will be added to traceability records after implementation.

12. Suspension & Resumption Criteria

Suspend testing if: the build does not configure or compile; the application crashes or enters an unsafe state in a way that invalidates further cases; test data/environment is inconsistent; or a blocking defect prevents execution of dependent tests.

Resume testing if: the blocking defect is fixed, the project builds cleanly, the test environment and fixture are stable, and a smoke run confirms that the affected path is available.

13. Test Case Management & Traceability

The RTM maps each SRS requirement to its planned test case. The current SRS RTM marks requirements N (not yet verified); all cases below are planned and have not been executed. Source-file references required by NFR-08 will be filled from the implementation when code exists; the SRS specifies module names but does not identify source files.

13.1 Requirements Traceability Matrix

Req. ID	Requirement	SRS module	Test ID	Level / status
FR-01	Create/load scenario	Scenario Module	TC-01	Unit, system; not executed
FR-02	Generate vehicles	Vehicle Generation Module	TC-02	Unit, system; not executed
FR-03	Assign vehicle attributes	Vehicle Module	TC-03	Unit; not executed
FR-04	Manage entry queues	Queue Management Module	TC-04	Unit, integration; not executed

Req. ID	Requirement	SRS module	Test ID	Level / status
FR-05	Maintain signal phases	Signal Control Module	TC-05	Unit, integration; not executed
FR-06	Advance signal timers safely	Signal Control Module	TC-06	Unit, system; not executed
FR-07	Adapt green duration	Adaptive Signal Module	TC-07	Unit, system; not executed
FR-08	Handle emergency priority	Emergency Priority Module	TC-08	Integration, system; not executed
FR-09	Represent road network	Road Network Module	TC-09	Unit; not executed
FR-10	Move vehicles through network	Vehicle Movement Module	TC-10	Integration, system; not executed
FR-11	Enforce downstream capacity	Vehicle Movement Module	TC-11	Unit, integration; not executed
FR-12	Select vehicle routes	Routing Module	TC-12	Unit; not executed
FR-13	Detect congestion	Congestion Detection Module	TC-13	Unit, system; not executed
FR-14	Simulate road incidents	Incident Management Module	TC-14	Unit, integration; not executed
FR-15	Respond to congestion/incidents	Traffic Management Module	TC-15	Integration, system; not executed
FR-16	Collect simulation metrics	Metrics Module	TC-16	Unit, system; not executed
FR-17	Export simulation report	Reporting Module	TC-17	System; not executed
FR-18	Ensure reproducibility	Simulation Controller	TC-18	System, regression; not executed
NFR-01	Maintainable module interfaces	All Modules	TC-NFR-01	Design/code review; not executed
NFR-02	Independent deterministic tests	All Modules	TC-NFR-02	Unit; not executed
NFR-03	Performance benchmark	Simulation Engine	TC-NFR-03	Performance; not executed
NFR-04	Reliable invalid-input handling	Scenario Module	TC-NFR-04	Negative, system; not executed
NFR-05	Linux C++17/CMake portability	Build System	TC-NFR-05	Build; not executed
NFR-06	Readable console and reports	Console Interface	TC-NFR-06	Usability; not executed
NFR-07	Security and robustness	Input Validation / Core	TC-NFR-07	Robustness, review; not executed
NFR-08	Requirement/source/ test traceability	Project Documentation	TC-NFR-08	Traceability review; not executed
NFR-09	Warnings, format, static analysis	Build and Analysis Tools	TC-NFR-09	Static/code review; not executed

Req. ID	Requirement	SRS module	Test ID	Level / status
NFR-10	Record and reuse random seed	Vehicle Generation / Simulation Controller	TC-NFR-10	System, regression; not executed

Status values: Not executed = planned and pending implementation; Pass/Fail/Blocked are assigned during execution. Defects and evidence are recorded in the test execution log.

13.2 Detailed Test Cases

For each case, record the build/revision and attach logs or screenshots where useful. Actual result and final status remain Not executed until a tester runs the case.

TC-01 | FR-01 | Load valid and invalid scenarios

Preconditions / data: Use one valid scenario and fixtures with a missing road endpoint, negative capacity, and invalid duration.

Steps: Load each fixture through the supported scenario interface.

Expected: The valid scenario loads; each invalid configuration is rejected with a clear error and no crash. Actual: Not executed. Status: Not executed.

TC-02 | FR-02 | Generate vehicles to a configured schedule

Preconditions / data: Load a small scenario with a deterministic vehicle schedule and known arrival ticks.

Steps: Start the simulation and inspect generated vehicles at each scheduled tick.

Expected: Vehicles are generated according to the configured schedule; no unscheduled generation occurs in this deterministic case. Actual: Not executed. Status: Not executed.

TC-03 | FR-03 | Assign unique and valid vehicle attributes

Preconditions / data: Configure multiple vehicles with valid sources, destinations, classes, and connected routes.

Steps: Generate the vehicles and inspect their identifiers and required attributes.

Expected: Every generated vehicle has a unique ID and the configured valid source, destination, class, and route data. Actual: Not executed. Status: Not executed.

TC-04 | FR-04 | Enforce entry-lane capacity

Preconditions / data: Set an entry lane capacity to 2 and schedule 3 vehicles for that lane.

Steps: Advance generation while inspecting the lane queue and handling of the third vehicle.

Expected: The queue never exceeds 2; the additional vehicle is not inserted beyond capacity and is handled safely. Actual: Not executed. Status: Not executed.

TC-05 | FR-05 | Maintain valid signal state per intersection

Preconditions / data: Load a scenario with controlled intersections and configured signal phases.

Steps: Initialize the simulation and inspect each intersection signal state.

Expected: Each controlled intersection has a valid phase/state defined by its configuration. Actual: Not executed. Status: Not executed.

TC-06 | FR-06 | Advance signal timers without conflicting greens

Preconditions / data: Configure an intersection with conflicting movements and valid phase timings.

Steps: Advance ticks through multiple transitions and inspect active phases and timers at each step.

Expected: Transitions follow configured timing rules and conflicting green phases are never active simultaneously. Actual: Not executed. Status: Not executed.

TC-07 | FR-07 | Adapt green timing within configured bounds

Preconditions / data: Prepare two otherwise comparable cases with different queue lengths and configured minimum/maximum green durations.

Steps: Run adaptive control and record the chosen durations and queue inputs.

Expected: Queue information affects the extension/shortening decision, and every selected duration remains within the configured minimum and maximum. Actual: Not executed. Status: Not executed.

TC-08 | FR-08 | Grant emergency priority safely

Preconditions / data: Create a valid emergency-priority request for a movement that conflicts with another approach.

Steps: Submit the request while the conflicting approach is active; observe signal handling.

Expected: The request triggers the configured priority behavior, while no conflicting green phase is enabled. Actual: Not executed. Status: Not executed.

TC-09 | FR-09 | Represent connected road network data

Preconditions / data: Load a small graph with known nodes, edges, capacities, and travel times.

Steps: Query or inspect the network through the road-network interface.

Expected: Configured connections, capacities, and travel times are accessible and match the scenario data. Actual: Not executed. Status: Not executed.

TC-10 | FR-10 | Move eligible vehicles along planned routes

Preconditions / data: Place a vehicle at a connected approach with a valid route, green signal, and downstream space.

Steps: Advance the simulation through the movement opportunity.

Expected: The vehicle advances along its planned connected route only when signal and capacity conditions permit. Actual: Not executed. Status: Not executed.

TC-11 | FR-11 | Block movement into a full downstream lane

Preconditions / data: Fill the downstream lane to its configured capacity and place an eligible vehicle upstream.

Steps: Advance the simulation while inspecting both queue occupancies.

Expected: The upstream vehicle does not enter the full downstream lane; occupancy never exceeds configured capacity. Actual: Not executed. Status: Not executed.

TC-12 | FR-12 | Select a minimum-cost connected route

Preconditions / data: Create a graph with at least two connected routes whose configured route costs are known and unequal.

Steps: Request a route to the destination using the configured shortest/weighted-path policy.

Expected: The selected route consists of connected edges and has the minimum configured route cost. Actual: Not executed. Status: Not executed.

TC-13 | FR-13 | Detect congestion at configured threshold

Preconditions / data: Configure a queue/occupancy threshold and prepare observations immediately below and at/above it.

Steps: Evaluate congestion for each observation.

Expected: Conditions below the configured threshold are not flagged; conditions meeting the threshold are flagged. Actual: Not executed. Status: Not executed.

TC-14 | FR-14 | Mark blocked road unavailable

Preconditions / data: Load a scenario containing a road segment used by a planned route.

Steps: Inject a road blockage and inspect road state and attempted movements through that segment.

Expected: The affected segment becomes unavailable or restricted and prohibited movement through it is prevented. Actual: Not executed. Status: Not executed.

TC-15 | FR-15 | Respond to incident with rerouting and no-route handling

Preconditions / data: Prepare one network with an alternate route and another where blockage removes all routes to the destination.

Steps: Trigger an incident or congestion response in both networks.

Expected: The configured response activates; the first vehicle is rerouted as supported, and the no-route case is handled safely without invalid movement or a crash. Actual: Not executed. Status: Not executed.

TC-16 | FR-16 | Collect traffic and trip metrics

Preconditions / data: Run a small deterministic scenario with known vehicle completions, waits, and queue observations.

Steps: Compare reported throughput, waiting time, queue statistics, and completed/unfinished trip counts against recorded events.

Expected: Each metric reflects the simulation events and final vehicle states for the scenario. Actual: Not executed. Status: Not executed.

TC-17 | FR-17 | Produce readable text or CSV summary

Preconditions / data: Complete a scenario with known metrics and select each supported output format.

Steps: Generate and open the final text and/or CSV report.

Expected: The report is readable and contains the required metrics; CSV records use consistent labels and valid row/column structure. Actual: Not executed. Status: Not executed.

TC-18 | FR-18 | Repeat equivalent simulation with the same seed

Preconditions / data: Use a fixed scenario configuration and deterministic seed.

Steps: Run the same simulation twice and compare generated traffic and final results.

Expected: Both runs produce equivalent results under deterministic execution conditions. Actual: Not executed. Status: Not executed.

TC-NFR-01 | NFR-01 | Verify clear module interfaces and controlled shared state

Preconditions / data: Review documented module boundaries and the integrated source when available.

Steps: Inspect interfaces and state ownership across modules, including mutable data shared between modules.

Expected: Module communication uses clear defined interfaces; uncontrolled global mutable state is absent outside the controlled simulation context. Actual: Not executed. Status: Not executed.

TC-NFR-02 | NFR-02 | Test each module independently and deterministically

Preconditions / data: Select deterministic public methods in each available module and prepare isolated fixtures.

Steps: Run module tests without starting the entire simulation; repeat equivalent input/output cases.

Expected: Each module can be tested independently and produces the expected deterministic outputs. Actual: Not executed. Status: Not executed.

TC-NFR-03 | NFR-03 | Run the standard performance scenario

Preconditions / data: Use a normal student laptop and a configured scenario with at least 4 intersections and 500 vehicles.

Steps: Run the scenario to completion; record hardware, build, seed, elapsed time, and any stalls.

Expected: The scenario completes without noticeable hangs; elapsed time and environment details are recorded for evaluation. The SRS defines no numeric time limit. Actual: Not executed. Status: Not executed.

TC-NFR-04 | NFR-04 | Reject invalid configuration safely

Preconditions / data: Prepare invalid capacities, impossible timings, and references to unknown roads/intersections.

Steps: Load or submit each invalid value through the supported interface.

Expected: Every invalid configuration is rejected with a clear error and without a crash or undefined behavior. Actual: Not executed. Status: Not executed.

TC-NFR-05 | NFR-05 | Build with C++17 and CMake on Linux

Preconditions / data: Use a clean Linux build directory and the project-supported C++17 compiler.

Steps: Configure with CMake and build the project using the documented commands.

Expected: The project configures and builds successfully as C++17 on Linux without relying on compiler-specific extensions. Actual: Not executed. Status: Not executed.

TC-NFR-06 | NFR-06 | Review console and report usability

Preconditions / data: Use the console menu/commands and inspect a representative simulation report.

Steps: Ask a reviewer unfamiliar with the run to locate key commands, understand an error, identify simulation status, and interpret metric labels.

Expected: Commands, errors, status, and report labels are readable, consistent, and interpretable. Actual: Not executed. Status: Not executed.

TC-NFR-07 | NFR-07 | Validate input handling and resource safety

Preconditions / data: Prepare malformed and boundary menu/file input, impossible values, and file-operation success/failure paths.

Steps: Exercise validation paths; inspect critical input code for unsafe C-style string use and resource management.

Expected: Invalid inputs are rejected safely; unsafe C-style string handling is absent; acquired resources are managed with RAII and released on success/failure. Actual: Not executed. Status: Not executed.

TC-NFR-08 | NFR-08 | Check requirement-to-source-to-test traceability

Preconditions / data: Use the implemented requirement list, source tree, and Section 13 RTM after implementation.

Steps: For each implemented FR, verify the requirement ID maps to source file(s) and test ID(s); note any unimplemented requirement explicitly.

Expected: Every implemented FR has a source-file reference and test ID in the RTM. Source references remain pending until implementation files exist. Actual: Not executed. Status: Not executed.

TC-NFR-09 | NFR-09 | Review warnings, formatting, and static analysis

Preconditions / data: Use the project build and the tools/commands selected by the development team.

Steps: Build with configured warnings; check formatting consistency; run static analysis and triage findings.

Expected: Warnings are enabled, formatting is consistent, and no critical static-analysis finding remains unresolved. Actual: Not executed. Status: Not executed.

TC-NFR-10 | NFR-10 | Record and reuse the random seed

Preconditions / data: Run a scenario that uses random vehicle generation with a specified seed.

Steps: Check the displayed/stored seed, then repeat with the same configuration and seed and compare generated traffic/results.

Expected: The seed used is printed or stored, and repeated runs with the same seed and configuration produce equivalent traffic and results. Actual: Not executed. Status: Not executed.

14. Test Metrics & Reporting

Metrics: Number and percentage of planned cases executed, passed, failed, blocked, and not executed; requirement coverage; open defects by severity and age; benchmark elapsed time and environment; reproducibility results; and number of requirements with pending source mappings.

Reports: Maintain execution evidence and defects as tests run. Provide a concise status report during execution and a final Test Summary Report that records build/configuration, coverage, results, open issues, and acceptance recommendation.

15. Approvals

Role	Name	Signature / Date
Test Plan owner	Aarya / Vainik	
Project reviewer	To be assigned	
Course evaluator / coordinator	To be assigned	